
TECHNICAL PLAN

EMS Build Guide

System design end to end, and the 20-day plan to build it. React + Express + PostgreSQL + Prisma, deployed on Hostinger VPS.

EMS — HR & Payroll Management System
CareerMap Solutions · Internal document

EMS — Build Guide

The one document you work from. System design end to end, then the 20-day plan to build it.

Project: HR & Payroll system for CareerMap Solutions **From:** React + Supabase · **To:** React + Express + PostgreSQL + Prisma **Built by:** Devesh, solo

Version 2. This guide was reviewed against the live repository and the npm registry by five specialist reviewers, who found 113 problems in version 1 — including a toolchain mismatch that would have stopped work on Day 2. Every correction is applied below.

Reference: [SYSTEM_DESIGN_AND_AUDIT.md](#) — the 438 problems in the current code · [ROADMAP.md](#) — strategy and the later SaaS phase

How to use this document

Part A — the system design. Read it fully before Day 0, then keep returning to it. **Part B** — the day-by-day plan. Read the day's section each morning. **Part C** — how you work: branching, routine, safety. **Part D** — reference: learning, environment, client questions, triage. **Part E** — what happens after Day 20.

The rule everything follows

Architecture quality is fixed. Days are fixed. Only scope moves.

A half-built module can be finished next week. A compromised architecture has to be rebuilt. Part D §4 states the exact order in which scope gives way.

PART A — SYSTEM DESIGN

A1. The stack

Layer	Choice	Pinned version	Why
Frontend	React + Vite	existing	Kept and adapted, not rewritten
API	Express	<code>express@^5</code>	Your own trust boundary
Database	PostgreSQL	17 (Neon)	Payroll data is deeply related

Layer	Choice	Pinned version	Why
ORM	Prisma	6.19.3 exactly	Typed queries, real migrations
Auth	Own JWT	jsonwebtoken , bcryptjs	Full control
Files	Own storage	S3-shaped — \$A12	Local in dev, Cloudflare R2 in production
Server language	TypeScript	with tsc --noEmit in CI	Catches the bug class that broke this product
Frontend language	JavaScript	unchanged	Converting 10,747 working lines is a rewrite we are avoiding

⚠ Versions are not optional

Your machine runs **Node v20.20.0**.

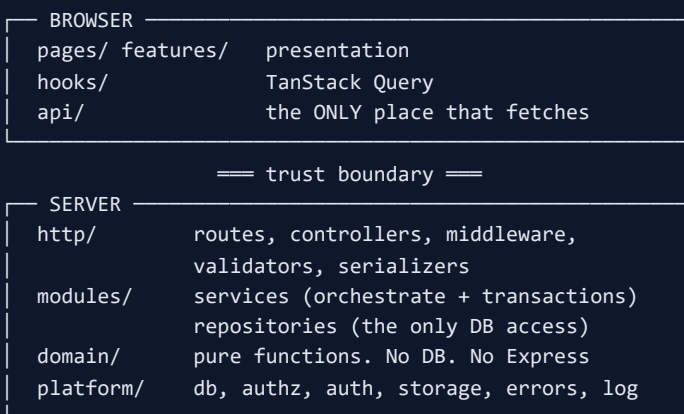
- `npm i prisma` today resolves to **8.0.0-rc**, which requires **Node ≥ 22.18** — the CLI will not start. And `@prisma/client` latest is 7.x, so you would also have a CLI/client major mismatch.
- Install **prisma@6.19.3** and **@prisma/client@6.19.3** — **the same exact version for both**. 6.19.3 needs only Node ≥ 18.18, has multi-file schema as a stable feature, and matches every tutorial you will find.
- `npm i express` gives **Express 5**, which is what we want — but Express 5 changed routing. `app.use('*', ...)` **throws at startup**. The catch-all is `app.use('/*splat*', notFound)`.
- **Never run bare** `npm run prisma` after install — it can fetch a different version. Use the local binary via npm scripts.

The three problems this architecture exists to solve

1. **Every business rule lives in the browser**. Fourteen rules with nothing behind them. Anyone with devtools can send what they like and the database accepts it.
2. **The same rule exists in many places and the copies disagree**. The salary formula exists as **seven copies across six files** — `Payroll.jsx:15-28` , `AddEmployeeModal.jsx:21-33` , `EmployeeDrawer.jsx:21-32` , `useEmployees.js:63-71` and `useEmployees.js:128-136` , `useReports.js:138-143` , `lib/supabase.js:449-457` .
3. **The app hides its own failures**. Zero error handlers anywhere. When something fails it invents data — fake paylips badged "Generated" (`Payroll.jsx:466`), a UAN built from the employee's PAN (`useReports.js:144`), a cancelled cheque the app draws itself (`BankVerificationModal.jsx:535-572`), a ₹4.1 lakh chart fallback (`useReports.js:188`), a "Saved!" toast that persists nothing.

Everything below makes these **structurally impossible**, not merely fixed once.

A2. Layers



Layer	Knows	Must never touch
http/	HTTP, status codes	business rules, the database
*.service	business steps, transactions	req , res , SQL
*.repository	Prisma queries	business rules, HTTP
domain/	numbers and dates	everything else

Direction is one-way: `http` → `modules` → `domain`. Never the reverse.

A3. Folder structure

```
EMS/
├── web/                                the React app (everything moves here on Day 1)
│   ├── index.html  package.json  vite.config.js  eslint.config.js
│   ├── vercel.json  public/  .env
│   └── src/
│       ├── api/          ← NEW. http.js + one file per module
│       ├── hooks/        ← bodies rewritten
│       └── pages/ features/ components/ stores/ utils/
└── server/
    ├── package.json          "prisma": { "schema": "prisma/schema" }
    ├── tsconfig.json
    ├── prisma/
    │   ├── schema/          base · identity · org · people · attendance
    │   │                   leave · payroll · documents · audit
    │   ├── migrations/      forward-only. never a reset script
    │   └── seed/ reference.ts  demo.ts
    ├── scripts/bootstrap.ts
    └── src/
        ├── main.ts  app.ts  config/env.ts
        ├── http/    routes/ controllers/ middleware/ validators/ serializers/
        ├── modules/ auth employees attendance leave payroll
        │                   documents notifications organization reports users
        │                   ↳ each: .service.ts .repository.ts .policy.ts
        ├── domain/   payroll/ leave/ attendance/ shared/
        ├── platform/ db/ authz/ auth/ storage/ errors/ logger/
        └── tests/     domain/ api/ authz/
```

Multi-file Prisma schema needs three things (none are defaults):

1. Prisma ≥ 6.7
2. `"prisma": { "schema": "prisma/schema" }` in `server/package.json`
3. Delete the `prisma/schema.prisma` that `prisma init` generates — you cannot have both, and the CLI errors if you do

Exactly one file (`base.prisma`) holds `generator` and `datasource` . Relations across files work with no imports.

A4. One request, traced end to end

A manager approves a leave request.

```

1. Leave.jsx          user clicks Approve
2. useLeave.js         api.leave.decide({ id, status: 'approved' })
                      no longer sends reviewed_by – the client does
                      not get to say who approved something
3. api/http.js        PATCH /api/leave-requests/:id/decision
    ===== network =====
4. requestContext     attach a request id
5. authenticate       verify JWT → load user → build ctx
                      REJECT if account is not active
6. validate          zod. unknown keys stripped
7. authorize          does this user hold 'leave:decide'?
8. controller         leaveService.decide(ctx, id, body) ← 5 lines
9. service – ONE transaction:
    a. load scoped to DIRECT_REPORTS
      not a report? → 404, NOT 403 (never leak that the row exists)
    b. domain/leave.assertDecidable(request, ctx, today)
    c. update WHERE status = 'pending'
      0 rows → 409 Already decided (no race)
    d. write LeaveLedgerEntry
    e. create on_leave attendance rows
    f. write audit row
    g. queue notification INSIDE the transaction
10. errorHandler      the ONLY file that sends 4xx/5xx
    ===== network =====
11. useLeave.js        success → invalidate cache
                      failure → MutationCache.onError toasts
                      automatically

```

A5. The rules that keep it correct

Enforced by lint and tests, not by remembering.

#	Rule
1	Only <code>platform/db/scoped.ts</code> and <code>unsafe.ts</code> may import the raw Prisma client
2	<code>ctx.db</code> appears only in <code>*.repository.ts</code>
3	<code>req</code> / <code>res</code> never appear below <code>http/</code>
4	<code>domain/</code> imports nothing from the project
5	Never compare <code>role === 'hr'</code> — only <code>can('payroll:run:create')</code>
6	Serializers are <code>allow-lists</code> (<code>pick</code> , never <code>omit</code>) and contain no <code>??</code> or <code>`</code> \ <code>`</code> fallback for business data
7	No statutory constant outside <code>domain/</code> — see below
8	No <code>toISOString()</code> outside <code>domain/shared/dates.ts</code>

Rule 7 in practice. Ban, with word boundaries, inside `server/src/**` except `domain/` : `\b\.``40\b` · `\b\.``50\b` · `\b\.``10\b` · `\b\.``12\b` · `\b\.``0075\b` · `\b\.``0325\b` · `\b\.``0833\b` · `\b\.``0367\b` · `\b2100\b` · `\b15000\b` · `\b1250\b` · `\b200\b` · `\b300\b`

A bare grep for `21000` also matches `210000`, and `15000` matches `150000` — word boundaries matter. Switch the rule on for `web/` only after Day 17, because the frontend copies live in files that are not touched until then.

Rule 8 in practice. Seven files derive "today" from UTC, and only two use `.split('T')[0]` — the rest use `.slice(0,10)`. Ban the `toISOString` identifier itself, not a suffix. The files: `useDashboard.js:16,75,137,192` · `Leave.jsx:297,399` · `Attendance.jsx:42,81,100` · `ApplyLeaveModal.jsx:54` · `Employees.jsx:100` · `Payroll.jsx:309` · `lib/supabase.js`.

A6. Data model

The identity split

User	login only: email, passwordHash (nullable), tokenVersion
Organization	the company. one row today, many later
Membership	User × Organization, carries the role
Employee	the HR record

This fixes three shipped bugs at once: **bootstrap becomes possible** (the first admin is a User with no Employee); **removing access stops destroying payslips**; and **role moves off the row an employee can edit** — today any employee can set their own role to `super_admin`.

Entities

Group	Entities
Identity	User · Organization · Membership · RefreshToken · PasswordResetToken
People	Employee · EmployeeFinancial · EmployeeBankAccount · EmployeeStatutoryIdentity · Department · Designation
Time	Attendance · Holiday · WorkCalendar
Leave	LeaveType · LeaveRequest · LeaveLedgerEntry
Money	SalaryStructure · PtSlab · EsiCoverage · EmployeeTdsDirective · PayrollRun · Payslip · PayslipLine
Files	EmployeeDocument · CompanyDocument · FileObject
System	OrganizationPolicy · Notification · AuditLog

`EmployeeStatutoryIdentity` holds **UAN, PF member id, ESIC number, PAN** — none of which exist in the current database, and all of which a payslip and every statutory filing require.

Decisions that matter

Decision	Reason
Money is integer paise on the server, rupees at the API boundary	Floats lose money. The conversion is pinned by a serializer test so the React screens keep working

Decision	Reason
Leave days are <code>Decimal(4,1)</code>	Half-days are structurally impossible today
Leave balance is a ledger , not a number	Entries: <code>+12 opening</code> , <code>-3 sick</code> , <code>+3 refund</code> . Balance is the sum. Kills the bug where rejecting a leave returned <i>more</i> days than it took, and it is the only way opening balances can be entered
Salary structure stores inputs only , effective-dated	Today a UNIQUE on employee makes salary history impossible
OrganizationPolicy is effective-dated too	A PF or PT rate change mid-year must not silently rewrite last month's payslip
Sensitive fields in separate tables	Row rules cannot hide columns
All timestamps <code>@db.Timestamptz(3)</code>	Prisma's default maps to <code>timestamp</code> without a zone. Date-only business keys stay <code>@db.Date</code>
Every tenant table has <code>organizationId</code> NOT NULL	Adding it after go-live is the most expensive SaaS migration there is
Every unique constraint includes <code>organizationId</code>	<code>@@unique([organizationId, employeeCode])</code>
Soft delete everywhere	Statutory records must survive

The schema is written fresh — five tables diverge, not two

- `profiles` is missing 9 bank columns
- `salary_structures` is missing 5, has `effective_from` NOT NULL with no default that the app never sends, and a bare UNIQUE on `employee_id` that blocks salary history
- `employee_documents` is missing `file_url` and `file_type`, both written on every upload
- `attendance` is missing the 4 geofence columns the modal sends
- `notifications` exists in two mutually incompatible definitions; `create_employee_account` is called with 8 parameters it does not declare

`docs/field-contract.json` must cover all five. It is a **Day 7 deliverable**, generated by grepping `web/src/pages` and `web/src/features` for field reads — pinned before the first module ships, not "checked in CI" one day.

A7. Authorization — four layers

Layer 1 — Company scope, automatic. `forOrg(orgId)` returns a Prisma client extension that injects `organizationId` into every query and create. `forOrg(undefined)` throws.

Honest limitation. A client extension cannot intercept `$queryRaw`, and it does not reach nested `connect:` writes. So scoping is *enforced by default, not unbypassable*. Therefore: `$queryRaw` and nested `connect` are banned by lint outside `platform/db`, and a test asserts the ban.

Layer 2 — Permission. `requirePermission('payroll:run:create')`. One registry maps roles to explicit permission sets. `super_admin` gets an enumerated list, never a wildcard.

Layer 3 — Data scope. `SELF · DIRECT_REPORTS · DEPARTMENT · ORGANIZATION`. Repository functions **require** a scope argument — omitting it is a compile error. **This applies to by-id reads too, not just lists.** `GET /employees/:id` without a scope filter is the classic IDOR hole. Out of scope always returns **404, never 403**. `DIRECT_REPORTS` needs `Employee.reportingManagerId` — it is in the model.

Layer 4 — Field visibility. Serializers are **allow-lists**. And the repository must not even fetch what the caller may not see:

```
employeeRepo.list(ctx, scope, {
  includeCompensation: ctx.can('employee:compensation:read'),
  includeBank:         ctx.can('employee:bank:read'),
  includeIdentity:      ctx.can('employee:identity:read'),
})
```

Three permissions, not one — salary, banking and tax identity are different data classes held by different roles.

Exports, reports and PDFs go through serializers too. They are the usual bypass.

Sessions

- **Access token:** in memory, 15 min. Carries `tokenVersion` as a claim
- **Refresh token:** `httpOnly; Secure; SameSite=Lax; Path=/api/auth; Max-Age=7d`
- **Rotation on every refresh.** `RefreshToken` stores `tokenHash` (sha256, never the raw token), `familyId`, `expiresAt`, `revokedAt`, `replacedById`. If a revoked token is presented, that is **reuse** → revoke the whole family, bump `tokenVersion`, audit, 401
- `tokenVersion` bumps on termination, password change and reuse detection

✅ **Resolved by the deployment target.** The client will host on **Hostinger VPS**, with Nginx serving the React build and reverse-proxying `/api` to Node on the same box. That makes the whole application **same-origin** — one domain, one certificate, no CORS, and `SameSite=Lax` works exactly as intended. This is a better setup than a split host (a Vercel URL plus a Railway URL would be cross-site and the refresh cookie would simply be dropped). Build against it: keep `CORS_ORIGIN` for local development only, and do not design around cross-origin cookies. Full deployment detail — VPS provisioning, Nginx, PM2, Certbot, firewall, backups — is written up closer to Day 20. The only thing that matters during the build is: **same origin, so do not add cross-site workarounds.**

CSRF: `/api/auth/refresh` and `/api/auth/logout` are the only cookie-authenticated routes. Both must require `Content-Type: application/json` and a custom header (`X-Requested-With: ems`) — a cross-site form cannot send either.

A8. Error contract

```
success { data, meta: { requestId } }  
failure { error: { code, message, requestId } }
```

`code` is a stable enum shared with the frontend.

`ValidationError` 422 · `AuthError` 401 · `PermissionError` 403 · `NotFoundError` 404 · `ConflictError` 409 ·
`BusinessRuleError` 422 · `InternalError` 500.

`errorHandler.ts` is the only file permitted to send a 4xx or 5xx. This works *because Express 5 forwards rejected promises from async handlers automatically*. On Express 4 it silently would not, and requests would hang.

Why fabricated data becomes impossible: the server never returns a placeholder. A missing payslip is a **404**, not a badge. An unissued UAN is **null**, not derived from the PAN. An unconfigured capability is a **blocking state**, not a zero.

Why silent failure becomes impossible: `api/http.js` throws → every hook errors by default ·

`QueryCache.onError` + `MutationCache.onError` toast by default · `<DataState>` cannot render success children while an error exists · on the server, `console.error` -and-continue is banned.

A9. Transactions

`withTransaction(ctx, fn)` in `platform/db/transaction.ts` .

Correction to a common assumption. Inside an interactive transaction Prisma hands back a client with `$extends` removed — you cannot simply re-wrap `tx` with `forOrg()` . Instead, `withTransaction` builds a `TxCtx` whose repositories receive `tx` plus the `orgId` explicitly, and every repository signature already takes `ctx` . A test asserts that no repository ever runs a query without an `organizationId` predicate.

Set the timeout. Prisma defaults to `timeout: 5000ms` , `maxWait: 2000ms` . A payroll run writing ~100 payslips will exceed that. Use `{ timeout: 120_000, maxWait: 10_000 }` for payroll and imports.

Nine atomic operations: payroll run creation · payroll state transition · leave decision · leave application · employee onboarding · termination · bank verification · attendance regularization · organization bootstrap.

Deliberately not atomic: reads, notification *delivery*, file bytes (write object, then row, with an orphan sweeper).

A10. The multi-tenant seam

In now, because each item is also a fix for a shipped bug:

`Organization` (where the fake "Saved!" tabs finally persist and the geofence leaves one admin's localStorage) ·
`organizationId` everywhere · `forOrg()` · `ctx` as the first argument · the User/Membership/Employee split ·
permissions never role names · storage keys `org/{orgId}/...`

Deferred: signup, company switching, platform console, impersonation, billing, branding, SSO.

What the SaaS phase touches later: `authenticate.ts`, the role registry, one new module, a signup route calling the existing `bootstrapOrganization()`. Zero repositories, zero services, zero domain functions, zero migrations, zero frontend call sites.

A11. Frontend architecture

`web/src/api/` — `http.js` handles base URL, auth header, envelope unwrapping, refresh-once-on-401, typed errors. One file per module beside it.

The naming convention — decide once, it governs every frontend day

For v1 the API emits `snake_case` and reproduces the join key the pages already read. A leave request serialises as `{ ...request, profiles: { full_name, employee_id, department } }`.

The database and server use camelCase; **serializers translate at the boundary**. Renaming the frontend to camelCase is a post-v1 task with its own day. Without this rule stated, every frontend day becomes a rename negotiation.

Hooks

Five hooks keep their exact signatures. Three do not, because scoping moves to the server:

- `useLeaveRequests(userId, role)` → `useLeaveRequests()` — forces an edit at `Leave.jsx:384`
- `useLeaveBalances(userId, role)` → `useLeaveBalances()` — `Leave.jsx:231`
- `useNotifications(userId)` → `useNotifications()` — `TopBar.jsx:20`, `NotificationPanel.jsx:76`

How much of the frontend really survives

Correction. Version 1 of this guide said "about 4,300 lines never change". That was wrong by roughly 4×.

- ~1,100 lines genuinely untouched: `Layout.jsx`, `TopBar.jsx`, `NotificationPanel.jsx`, `HRDashboard.jsx`, `EmployeeDashboard.jsx`, `Dashboard.jsx`, `main.jsx`
- ~5,900 lines keep their structure but need surgical edits across 14 page and feature files — salary formula removal, the hardcoded month picker, the Simulate GPS button, date helpers, CSV quoting, fabricated columns
- The rest is the hooks and the 5 Supabase-importing files, which are rewritten

The architecture still saves you most of the UI work. It does not save you from touching those 14 files, and no day should discover them by surprise.

A12. File storage

The app stores Aadhaar and PAN scans, cancelled cheques and payslip PDFs — sensitive documents and statutory records kept for years.

A VPS has a persistent disk, so `local.ts` is viable in production too — unlike Railway, Render, Heroku or Vercel, whose ephemeral filesystems wipe on every restart. The catch is that durability becomes yours: uploads must be in the daily offsite backup alongside the database dump, or a dead VPS takes every Aadhaar scan and payslip with it.

Build both implementations anyway. The interface costs nothing, and moving to R2 later is one environment variable.

```
interface StorageService {
  put(key, buffer, mimeType): Promise<void>
  get(key): Promise<Buffer>
  delete(key): Promise<void>
  exists(key): Promise<boolean>
}
```

`local.ts` for development, `r2.ts` for production, selected by one env var. No other file ever knows where a file physically lives.

Cloudflare R2, not AWS S3 — R2 charges nothing for downloads, and employees download payslips constantly. 10 GB free, S3-compatible so the same `@aws-sdk/client-s3` works.

Files are never served by a public URL

```
GET /api/employee-documents/:id/file
  → authenticate → load row, scoped → permission check
  → storage.get(key) → stream
```

Required response headers: `Content-Disposition: attachment; filename="..."` · `Content-Type` from a server-side allow-list, never the stored value · `X-Content-Type-Options: nosniff` . Without these, an uploaded HTML file becomes stored XSS on your own domain.

Payslip PDFs get the same treatment — they are the more sensitive object, not the less.

Keys: `org/{orgId}/employees/{employeeId}/{documentId}` — never user-supplied, so path traversal is impossible.

Payslips are immutable: stored once a run is `paid` , content-hashed, never deleted, never regenerated on demand.

Uploads validated: MIME allow-list and extension both checked · 5 MB cap · filename always replaced by a generated key.

A13. Testing

Install the runner on Day 3, not Day 20: `npm i -D vitest supertest @types/supertest` .

Layer	What	Built on
<code>domain/</code> unit	salary, statutory, leave days, geofence, state machines	as each is written
Golden payroll	~30 employees × real scenarios, signed by the accountant	Day 14
API integration	one allowed + one denied path per endpoint	as each module ships
Authorization matrix	every role × every resource	Day 7 onward
Tenant conformance	Prisma metadata: every table has <code>organizationId</code>	Day 2

The authorization matrix comes from `Role_Permission_Documentation.md` §3.1 (8 modules × 7 roles) and §3.2 (16 actions) — not §10, which is a manual smoke checklist worth running on Day 20.

The golden test must not be 30 plain full-month salaries. It must include: a mid-month joiner, a mid-month leaver, an LOP month, a February Maharashtra PT case, an employee crossing ₹21,000 mid-period (ESI stays locked), an employee at exactly ₹15,000 PF wages, and a zero-TDS directive.

PART B — THE 20 DAY PLAN

DAY 0 — Setup (the evening before, ~3 hours)

Nothing here is coding, and skipping it costs you Day 1 and Day 2.

Task	Detail
Rotate the leaked password	<code>EMS@2026</code> is hardcoded at <code>lib/supabase.js:353,368</code> , sits in the committed <code>dist/</code> bundle, and is written in plain text at <code>Role_Permission_Documentation.md:27</code> — which is pushed to GitHub. Change it in Supabase, strip it from the doc, delete the committed <code>dist/</code>
Neon	Account → project <code>ems</code> → databases <code>ems_dev</code> and <code>ems_demo</code> . Copy both connection strings for each: the pooled one and the direct (non-pooler) one
Cloudflare R2	Create the bucket now. Ten minutes, and Day 20 will not discover a storage problem
Confirm the Hostinger plan	Must be VPS (Ubuntu, root access). Shared hosting is PHP-oriented and cannot run Node, PostgreSQL or background jobs. If the client bought shared, raise it now — not on Day 20
Tools	DBeaver or Prisma Studio · VS Code Prisma extension · Thunder Client
Send the client questions	Part D §D3. These gate Days 9, 12 and 14
Ask the accountant	The signed salary sheet described in §A13

PHASE 1 — Foundation · Days 1–3

Day 1 — Structure and the Express skeleton

Move the whole frontend, not just `src/`. The repo root holds `index.html` (which loads `/src/main.jsx`), `package.json`, `vite.config.js`, `eslint.config.js`, `vercel.json`, `public/` and `.env`. Moving only `src/` breaks everything.

```
mkdir web
git mv src index.html package.json package-lock.json \
    vite.config.js eslint.config.js vercel.json public web/
mv .env web/.env # gitignored, so git mv will not see it
rm -rf node_modules
cd web && npm install && npm run dev # VERIFY the app still runs
```

Only when the frontend runs, create the server:

```
cd .. && mkdir server && cd server && npm init -y
npm i express@^5 zod dotenv cors helmet cookie-parser
npm i -D typescript tsx @types/node @types/express@^5 \
    @types/cors @types/cookie-parser vitest supertest @types/supertest
npx tsc --init
```

Build — `app.ts` (assembly, no listen) separate from `main.ts` (listen + graceful shutdown + `prisma.$disconnect`). `config/env.ts` reads `process.env` through zod and **throws at boot** — and validates *shape*, not just presence: JWT secrets must be ≥32 chars, `DATABASE_URL` must parse as a URL. Nothing else in the codebase ever reads `process.env`. Load `dotenv` as the very first import in `main.ts`. One route: `GET /health`.

Scripts: `"dev": "tsx watch src/main.ts"`, `"typecheck": "tsc --noEmit"`, `"test": "vitest"`.

tsx does not type-check — it strips types with esbuild. Run `npm run typecheck` before every commit, or TypeScript catches nothing for 20 days.

Express 5 note: the catch-all is `app.use('/*splat', notFound)`. Plain `/*` throws at startup.

Done when — `cd web && npm run dev` serves the app, and `localhost:4000/health` returns ok.

Day 2 — Database and first migration

```
cd server
npm i -D prisma@6.19.3
npm i @prisma/client@6.19.3
npx prisma init --datasource-provider postgresql
rm prisma/schema.prisma # you cannot have both this and schema/
mkdir prisma/schema
```

Add to `server/package.json`: `"prisma": { "schema": "prisma/schema" }` and `"postinstall": "prisma generate"`.

base.prisma :

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // POOLED — used at runtime
  directUrl = env("DIRECT_URL")   // DIRECT — used by migrate/studio
}
```

This is the single most common Prisma + Neon failure. Neon's default connection string is the **pooled** one (host contains `-pooler`). Prisma Migrate needs session-level advisory locks and cannot run through PgBouncer — `migrate dev` fails with advisory-lock errors that explain nothing. `DATABASE_URL` = pooled, with `?sslmode=require&pgbouncer=true&connect_timeout=15`. `DIRECT_URL` = unpooled. Neon free-tier computes also auto-suspend, so the first request after idle takes 5–10s — hence the raised timeout.

Build — `Organization`, `User`, `Membership`, `Employee`. `organizationId` on tenant models, org-prefixed unique constraints, all timestamps `@db.Timestamptz(3)`. Then `npx prisma migrate dev --name init`. Write the **tenant conformance test** today — ~40 lines reading Prisma's metadata, failing if a new table lacks `organizationId`.

Learn — the biggest new concept from MongoDB. Open the generated `migration.sql` and read it. That file is your database structure, it lives in Git, and it is how any machine rebuilds the same database.

Done when — Studio shows the tables, the conformance test passes, and `prisma/migrations/` has one committed folder.

Day 3 — Platform layer

Build — `platform/db/` (prisma singleton, `scoped.ts`, `transaction.ts` with the timeout settings from §A9) · `platform/errors/AppError.ts` · `http/middleware/errorHandler.ts` — the only file that sends 4xx/5xx · `requestContext.ts` · structured logger · the response envelope · `platform/storage/` interface + `local.ts` (~60 lines, belongs with platform, not Day 18) · `platform/auth/password.ts` with `bcryptjs` (needed today by bootstrap; `bcryptjs` avoids native build problems on Windows).

`scripts/bootstrap.ts` — creates Organization, policy rows, reference data, the first User and its Membership. It reads the admin password from an env var or prompts; it never hardcodes one, and it refuses to run if an Organization already exists.

Also add `prisma.seed` config so `migrate reset` actually reseeds:

```
"prisma": { "schema": "prisma/schema", "seed": "tsx prisma/seed/reference.ts" }
```

Done when — `npm run bootstrap` creates a company and an admin in an empty database.

→ merge · tag `v0.1-foundation`

PHASE 2 — Auth · Days 4–6

Day 4 — Login

Build — `jwt.ts`, `auth.service.login()` : resolve email-or-employee-code → verify password → **reject if status is not active** → issue access (15 min, carrying `tokenVersion`) + refresh. Set the refresh cookie exactly as §A7 specifies.

Done when — login returns `{ data: { accessToken, user } }` in the body and a `Set-Cookie` header. The refresh token never appears in a body, a log, or a URL.

Day 5 — Sessions and rotation

Build — `RefreshToken` model and full rotation per §A7, including **reuse detection**. `/refresh`, `/logout`, `/session` (user + employee + role + permissions in one response), `/change-password` (verifies the current password). `authenticate` middleware. Rate limit auth routes today, not on Day 20.

Done when — presenting a rotated-away refresh token revokes the whole family and forces re-login.

Day 6 — Permissions + first frontend cutover

Build (server) — `platform/authz/roles.ts` registry · `authorize` middleware · `scope.ts` · the authorization matrix test skeleton.

Build (web) — `api/http.js`, `api/auth.js`, `authStore` with `can()`, `App.jsx` session bootstrap, `SignIn.jsx`, `ProtectedRoute` on permissions, and `QueryCache.onError` + `MutationCache.onError` — the one change that fixes "zero onError handlers in the entire codebase".

Done when — you log into the real UI with your own account. Supabase Auth deleted.

→ merge · tag `v0.2-auth`

PHASE 3 — Employees and settings · Days 7–10

(Four days, not three. Day 9 in version 1 held three days of work.)

Day 7 — Employee reads + the field contract

Build — `Employee`, `EmployeeFinancial`, `EmployeeBankAccount`, `EmployeeStatutoryIdentity`, `Department`, `Designation`, `LeaveType`, `LeaveLedgerEntry` — **models and migration only**, so Day 8 can reference them.

Employee repository/service/controller/routes. `GET /employees`, `GET /employees/:id` — **both scoped**. Allow-list serializers with the three permissions from §A7.

Also today: generate `docs/field-contract.json` by grepping the pages for field reads. Pin it before any module ships.

Done when — two roles get different fields from the same endpoint, and the contract file exists.

Day 8 — Employee writes + user management

Build — `POST /employees` in one transaction. `PATCH /employees/:id` whose body has **no role, no status, no ctc**.

And the four capabilities the Supabase edge functions currently provide, which nothing else replaces: `PUT /memberships/:id/role` (permission `membership:role:assign`, `super_admin` only) · `POST /users/invite` · `PATCH /users/:id/status` · `DELETE /users/:id` (the **termination transaction** from §A9 — revokes access, keeps payslips).

Role assignment enforces three invariants, each tested: you cannot change your own role · you cannot grant a role broader than your own · the last active `super_admin` cannot be demoted.

Rewire `useUsers.js` and the Settings → Users tab.

Done when — creating an employee with a bad field leaves **zero** rows, and an employee cannot change their own role by any route.

Day 9 — Settings and master data

Build — `OrganizationPolicy` (effective-dated), `PtSlab` table, `Holiday`. Endpoints for company identity (**all 14 fields the Company tab currently discards**), geofence, statutory rates, leave types. `seed/reference.ts` including Maharashtra PT slabs. Wire the Settings page.

Done when — every Settings tab persists and survives a reload.

Day 10 — CSV import

Build — `POST /employees/import` with a dry-run preview. Rows parse through the **same zod schema** as `POST /employees` — no role, status, ctc or password field. 1 MB / 500 row cap.

Imported employees get no password. `passwordHash = null`, `status = 'invited'`, plus a single-use `PasswordResetToken` (hashed, 72h) that HR distributes. A User with a null hash cannot log in.

Done when — you import the client's real roster and none of them can log in until they set a password.

→ merge · tag `v0.3-employees`

PHASE 4 — Attendance and leave · Days 11–14

Day 11 — Attendance + dashboard part one

Build — `Attendance` with the four geofence columns. `domain/shared/dates.ts` → `zonedToday(now, timezone)` and the Rule 8 lint ban. Server-side geofence. **Delete the "Simulate GPS inside office" button** (`MarkAttendanceModal.jsx:172-179`) and make the geofence **fail closed** on GPS denial. `GET /dashboard/summary` attendance figures, aggregated in Postgres.

Day 12 — Attendance views + frontend

Build — monthly view (fixes the `-31` bug), hours worked across midnight, regularization. Rewire `useAttendance.js`.

Day 13 — Leave

Build — `LeaveRequest` + ledger. `domain/leave/leaveDays.ts` counting working days **excluding weekends and holidays**. `POST /leave-requests` and `/preview`. **Backfill opening ledger entries** for the imported employees.

Day 14 — Leave approval + dashboard

Build — approve/reject in one transaction, `DIRECT_REPORTS` scope, 404-not-403. Rewire `useLeave.js` — and note the three hook signature changes from `SA11`. Finish `GET /dashboard/summary` and `/dashboard/me`, then rewire `useDashboard.js` and **both dashboard components** — the first screen every role sees, and the only hook with no day in version 1. Delete its hardcoded `12/12/18/24/5` balance fallback (`useDashboard.js:164-171`).

→ merge · tag `v0.4-attendance-leave`

PHASE 5 — Payroll · Days 15–18

The highest-risk phase. Money bugs destroy trust permanently.

Day 15 — The salary engine and statutory rules

Build — `domain/payroll/salary.ts`, one function replacing all seven copies.

PF — PF wages = basic + DA + retaining allowance (per the 2019 Supreme Court ruling, any allowance ordinarily and universally paid counts — confirm the component list with the accountant). Employee 12%. Employer 12% of the same base, split: **EPS = 8.33% of min(PF wages, 15000), capped at ₹1,250; employer EPF = employer total – EPS**. An employee who first joined on or after 1 Sep 2014 with PF wages above ₹15,000 and no prior membership is **not** an EPS member — the full employer 12% goes to EPF.

ESI — **eligibility is locked per CONTRIBUTION PERIOD, not per month**. Periods are 1 Apr–30 Sep and 1 Oct–31 Mar. Store `EsiCoverage {employeeId, periodStart, periodEnd, covered, lockedWageRate, reason}`. The test runs once, at period start or date of joining, against the **wage rate** — not the amount actually paid. Once covered, contributions continue to period end even if wages later exceed ₹21,000. Employee 0.75%, employer 3.25%, each rounded **up** to the next rupee independently.

The current code re-evaluates ESI every month (`Payroll.jsx:23`). That is the bug.

PT is per-state, per-employee — keyed to where the employee physically works, not one company setting.

`PtSlab {state, effectiveFrom, gender, wageFrom, wageTo, amount, februaryAmount}`. Maharashtra: men — nil to 7,500, ₹175 to 10,000, ₹200 above, **₹300 in February**; women — nil to ₹25,000, then ₹200 / ₹300. Add an invariant test that annual PT never exceeds ₹2,500.

Golden test per `SA13`.

Done when — the golden test passes and all seven copies are deleted. *(The frontend copies live in files not rewired until Day 18 — delete them together and accept two days of a stale preview, or leave the frontend copies until Day 18 and enable the Rule 7 lint then.)*

Day 16 — Payroll runs, LOP and TDS

Define `paidDays` precisely — this is the most contested calculation in Indian payroll and four words will not do:

`paidDays` = calendar days in the employee's employment window within the month – `lopDays`, where the window runs from `max(joiningDate, monthStart)` to `min(lastWorkingDate, monthEnd)`. Weekly offs and declared holidays inside the window **count as paid**. Per-day rate = `monthlyGross / X`, where X is `OrganizationPolicy.lopBasis` ∈ {calendarDaysInMonth, fixed30, workingDaysInMonth} — **snapshotted onto the payslip**. Sandwich rule is a policy flag.

Thresholds test the wage rate; percentages apply to earned wages. Never test a threshold against an LOP-reduced figure. Store `ncpDays` (non-contributing period days) on the payslip — the EPFO ECR file requires it per member per month.

TDS — manual mode for v1. "Configured" means the accountant enters `EmployeeTdsDirective {employeeId, financialYear, monthlyAmount, enteredBy, effectiveFrom}` per active employee, including an explicit ₹0 with a reason. The run unblocks when every active employee has one. This is legal, auditable, and is what actually lets the client go live. A computed TDS engine with declarations and Form 16 is **explicitly out of v1 scope** — named here so nobody assumes it exists.

Build — `POST /payroll-runs` creating run + payslips + audit in one transaction with the raised timeout.

Day 17 — Payslips and PDF

Build — state machine enforced server-side, totals recomputed, locked after approval. `npm i pdfkit @types/pdfkit` — no browser needed, streams to a Buffer. Payslips stored to R2 with a content hash, served through an authenticated route. `GET /payslips/me`.

An Indian payslip must show: employer name and address, employee name and code, designation, **UAN**, **PF member id**, **ESIC number**, **PAN**, pay period, paid days and LOP days, each earning and deduction as a line, gross, total deductions, net in figures and words, and the employer's PF/ESI contributions.

Day 18 — Payroll frontend

Build — rewire `usePayroll.js`, payroll page, payslip modal, employee payslip view. Remove the fabricated statutory identifiers and the fake "Generated" badges. **Bank transfer advice file** — the client cannot pay anyone without it.

→ merge · tag `v0.5-payroll`

PHASE 6 — Remaining modules · Day 19

Build — `r2.ts` + bucket + upload validation + the download route with its three security headers. Documents verification (an employee can no longer verify their own). Notifications: real table, fan-out by **role query** not the hardcoded `demo-hr-admin-id` strings, polling.

Report endpoints aggregating in **Postgres**. **One shared `csv()` helper** — all six current exporters share a quote-escaping bug, have no BOM and no formula-injection guard.

Then **delete** `web/src/lib/supabase.js` and drop `@supabase/supabase-js`.

Done when — `grep -rn supabase web/src/ server/src/` returns nothing and the package is gone from `web/package.json`. (Version 1's test was `grep -r supabase src/`, which passes vacuously after Day 1's move.)

PHASE 7 — Finish · Day 20

Build — `<DataState>`, error boundaries, real empty states, replace `alert()` / `confirm()`. Audit log covering login success and failure, permission denials, role changes, salary changes, payroll transitions, document access and exports — not just the four in version 1. Security headers. Backup and a tested restore — actually restore it. Deploy. Smoke test every page as every role using `Role_Permission_Documentation.md §10`.

→ merge · tag `v1.0`

When a day overruns

It will. The rule:

1. Finish the server side, push it, leave the frontend rewiring. A module with a working API and an old UI is recoverable; the reverse is not.
2. Never carry an unmerged branch across two phase boundaries. Merge what works, open a follow-up.
3. Write the unfinished item into the next day's notes immediately — not "I'll remember".
4. If a day slips twice, take it from Part D §4's triage list rather than compressing the next day.

Days 12, 19 and 20 have the most slack. Days 15 and 16 have none — protect them.

PART C — HOW YOU WORK

C1. Branching

Keep `main`. Do not rename to `master` — GitHub's default is already `main` and renaming gains nothing.

```
main          always working
└─ phase/1-foundation  one branch per phase
```

```
git checkout main && git pull
git checkout -b phase/2-auth
# build, committing often
git push -u origin phase/2-auth
# open PR, read your own diff, merge
git checkout main && git pull
git tag v0.2-auth && git push --tags
```

Why branch alone? The tag is a rescue point (`git checkout v0.3-employees` and you are back on solid ground) · reading your own diff catches real bugs · `main` stays demoable.

C2. Daily routine

Morning — note the directories; after Day 1 there are two `package.json` files and Prisma lives only under `server/` :

```
# terminal 1
cd d:/CMS/EMS/server
npx prisma migrate deploy && npx prisma generate      # only after a pull that brought migrations
npm run dev

# terminal 2
cd d:/CMS/EMS/web
npm run dev
```

Then write **one sentence** for what "done" means today. On paper.

Every time something works — `git add . && git commit` . Ten to twenty times a day.

Before you stop — `npm run typecheck` , then `git push` even if unfinished. Two lines in a notes file: what got done, what to pick up tomorrow.

C3. Keeping your work safe

Practice	Why
Commit after every working piece	Undo one mistake, not a day
Push every evening	A dead laptop costs hours, not weeks
<code>npm run typecheck</code> before committing	<code>tsx</code> does not type-check
Tag after each phase	A known-good point
Read your own PR diff	You will find real bugs
Never <code>--force</code> push on <code>main</code>	Unrecoverable
Demo database stays separate	Development must never break the demo

Broke something badly — `git stash` → `git checkout v0.N-name` → diagnose → `git stash pop` .

When a migration goes wrong in development — `npx prisma migrate reset` rebuilds from migrations plus seed. It is only your database. **Never run it against demo or production.**

C4. Databases

Database	For
ems_dev	daily work. Break it freely
ems_demo	demos. Never experiment
later ems_prod	real use

Structure syncs through Git; data does not — and that is the point. One migration file applies to any database. Reference data travels as code through `seed/reference.ts`. Only test data differs, deliberately.

```
cd server
DATABASE_URL="<demo pooled>" DIRECT_URL="<demo direct>" npx prisma migrate deploy
```

PART D — REFERENCE

D1. What you already know

	Status
Express, middleware, controllers, JWT, React, REST	✓ same as MERN
Prisma queries	Mongoose-like. One day
Migrations	NEW Days 2–3
Transactions	NEW Day 8
Relational modelling	NEW Days 2, 7, 13
Layered architecture	NEW the most valuable thing here

Mongoose → Prisma

	Mongoose	Prisma
Find all	<code>Model.find({})</code>	<code>prisma.model.findMany()</code>
Filter	<code>.find({ status: 'active' })</code>	<code>findMany({ where: { status: 'active' } })</code>
Find one	<code>.findById(id)</code>	<code>findUnique({ where: { id } })</code>
Create	<code>new Model({}).save()</code>	<code>create({ data: { } })</code>

	Mongoose	Prisma
Update	<code>.findByIdAndUpdate()</code>	<code>update({ where, data })</code>
Relations	<code>.populate('dept')</code>	<code>include: { dept: true }</code>
Transaction	needs a replica set	<code>prisma.\$transaction(async tx => {})</code>
Migrations	none	real, versioned, in Git

D2. Environment variables

`server/.env` — never in Git

```

DATABASE_URL="postgresql://...-pooler.../ems_dev?sslmode=require&pgbouncer=true&connect_timeout=15"
DIRECT_URL="postgresql://.../ems_dev?sslmode=require"

JWT_ACCESS_SECRET="at least 32 characters"
JWT_REFRESH_SECRET="a different 32+ character string"
JWT_ACCESS_EXPIRY="15m"
JWT_REFRESH_EXPIRY="7d"

PORT=4000
NODE_ENV=development
CORS_ORIGIN="http://localhost:5173"
COOKIE_DOMAIN="localhost"

STORAGE_DRIVER="local"
STORAGE_PATH="./uploads"
# STORAGE_DRIVER="r2"
# R2_ACCOUNT_ID="" R2_ACCESS_KEY_ID="" R2_SECRET_ACCESS_KEY="" R2_BUCKET=""

BOOTSTRAP_ADMIN_EMAIL=""
BOOTSTRAP_ADMIN_PASSWORD=""

```

`web/.env`

```
VITE_API_URL="http://localhost:4000/api"
```

● Everything that reaches the frontend bundle is public — env vars, hardcoded literals, seed data, comments. Not just `VITE_` variables. Proved on this project: `EMS@2026` is **not** a `VITE_` variable — it is a hardcoded fallback at `lib/supabase.js:353,368`, and it is sitting in `dist/assets/index-DP8jYyHS.js` beside `admin@careermap.in`. It is also committed in plain text in `Role_Permission_Documentation.md:27`, pushed to GitHub. The lesson is not "avoid `VITE_` secrets". It is **never write a credential anywhere in `web/`**.

Commit `.env.example` for both, with empty values.

D3. Ask the client on Day 0

PF — (a) Is the establishment covered under the EPF Act? Give me the establishment code and ECR credentials. (b) Does the employer restrict contributions to the ₹15,000 ceiling, or contribute on actual PF wages? (c) For each existing employee: were they an EPF member before joining? (d) Which salary components count as PF wages? (e) Is EDLI applicable?

ESI — (f) Is the establishment ESI-covered, and under which sub-code? (g) Confirm the contribution-period rule is understood: eligibility locks for April–September and October–March.

PT — (h) In which **states** do employees physically work? I need one registration number per state.

TDS — (i) Who computes TDS today, and will they supply a monthly amount per employee? (v1 is manual mode.)

CTC — (j) Does the CTC figure include employer PF, employer ESI and gratuity? If yes, gross is **not** $\text{CTC} \div 12$, and every payslip changes.

Leave — (k) Is the leave year January–December or April–March? (l) Which types carry forward, to what cap, and does anything lapse or get encashed? (m) What are the opening balances for every employee?

Policy — (n) Can a manager approve anyone, or only direct reports? (o) Are **manager** and **rm** genuinely different roles? (p) Is the geofence a hard block or a recorded flag?

Data — (q) **Has anyone entered real employee data or uploaded real documents into the current system?** The audit indicates no — no first admin was ever created and Add Employee has never worked — in which case there is nothing to migrate and the new database starts empty. Confirm before Day 19 deletes the old client.

D4. If you fall behind — what gives way, in order

1. **Report screens** (Day 19) — but **not** the shared **csv()** helper. The current exporters are broken and formula-injectable; ship the helper even if the screens slip
2. **PDF payslips** (Day 17) — a clean print stylesheet is an acceptable interim, but the **bank transfer file is not optional** — without it nobody gets paid
3. **Documents module** (Day 19) — compliance, not payroll
4. **Notifications** (Day 19)
5. **Audit log UI** (Day 20) — keep *writing* audit rows; build the reading screen later

Never give way on: permissions and data scoping · transactions on payroll and employee creation · the single salary engine and its golden test · migration discipline · the error contract · the bank transfer file.

A missing module is a scheduling problem. A wrong salary or a leaked bank account is a trust problem — and trust does not come back.

D5. Commands

```
# morning – two terminals
cd server && npx prisma migrate deploy && npx prisma generate && npm run dev
cd web && npm run dev

# schema change (in server/)
npx prisma migrate dev --name what_changed && npx prisma generate

npx prisma studio           # look at data
npx prisma migrate reset    # dev database only
npm run typecheck           # tsx does NOT do this for you
npm test

# end of phase
git push -u origin phase/N-name # PR, read diff, merge
git checkout main && git pull && git tag v0.N-name && git push --tags
```

PART E — AFTER DAY 20

Tagging `v1.0` is not go-live. Budget one more week.

Step	Why
Parallel run for one month	Run the new payroll alongside however salary is calculated today, and reconcile every employee to the rupee. This is the only real proof
Client data load	Real roster, opening leave balances, statutory identities, bank details, salary structures — into <code>ems_prod</code> , not <code>ems_demo</code>
HR training	Two sessions: daily use, and the monthly payroll run
A written runbook	How to run payroll, what to do when it fails, who to call
Then switch off the old system	Not before the parallel month reconciles

Items explicitly deferred past v1, so nobody assumes they exist: computed TDS with declarations and Form 16 · Form 24Q and ECR file generation · camelCase API migration · signed URLs for files · the SaaS phase in [ROADMAP.md](#).

Day 0 begins with this

1. Rotate `EMS@2026` and delete the committed `dist/`
2. Send the Part D §D3 questions — they gate Days 9, 13 and 15
3. Ask the accountant for the signed salary sheet

4. Neon, R2, and the domain decision

Then Day 1: move the frontend, verify it still runs, and start the server.